TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A PROJECT REPORT

ON

UPTIME TRACKER: WEBSITE MONITORING AND AVAILABILITY TRACKING SYSTEM

Submitted By:

Abhishek Panthee	(THA080BCT002)
Alin Timelsana	(THA080BCT003)
Kiran Paudel	(THA080BCT019)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

In partial fulfillment for the award of the

Bachelors Degree in Electronics, Communication and Information Engineering

Under the Supervision of

Er. Rajad Shakya

December 2025

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A PROJECT REPORT

ON

UPTIME TRACKER: WEBSITE MONITORING AND AVAILABILITY TRACKING SYSTEM

Submitted By:

Abhishek Panthee	(THA080BCT002)
Alin Timelsana	(THA080BCT003)
Kiran Paudel	(THA080BCT019)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

December 2025

DECLARATION

We hereby declare that this project entitled **"UPTIME TRACKER: WEBSITE MONITORING AND AVAILABILITY TRACKING SYSTEM"** is our own original work. This project has been prepared and submitted as a DBMS academic project to the Department of Electronics and Computer Engineering, Thapathali Campus. The work presented in this report is genuine and has not been copied from any other source. This work has not been submitted elsewhere for academic evaluation or any other purpose.

Abhishek Panthee	(THA080BCT002)
Alin Timelsana	(THA080BCT003)
Kiran Paudel	(THA080BCT019)

Date: December 2025

COPYRIGHT

The author has agreed that the library, Department of Electronics and Computer Engineering, Thapathali Campus, may make this report freely available for inspection. Moreover, the author has agreed that the permission for extensive copying of this project work for scholarly purpose may be granted by the professor/lecturer, who supervised the project work recorded herein or, in their absence, by the head of the department. It is understood that the recognition will be given to the author of this report and to the Department of Electronics and Computer Engineering, IOE, Thapathali Campus in any use of the material of this report. Copying of publication or other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, IOE, Thapathali Campus and author's written permission is prohibited.

Request for permission to copy or to make any use of the material in this project in whole or part should be addressed to department of Electronics and Computer Engineering, IOE, Thapathali Campus.

ACKNOWLEDGEMENT

We would like to thank Institute of Engineering, Tribhuvan University, for incorporating major project within the curriculum of Bachelor's in Electronics, Communication, and Information Engineering.

Special thanks to our supervisor, **Er. Rajad Shakya**, for his guidance, invaluable feedback, and constant support for the project. His expertise and encouragement have been pivotal in shaping this project. We are also grateful to our project coordinator, **Er. Umesh Kanta Ghimire**, for his continuous support and guidance.

We would like to express our appreciation to the Department of Electronics and Computer Engineering, Thapathali Campus, for their continuous assistance, recommendations during project selection. This has significantly helped in enabling our professional development.

We extend our sincere thanks to all the teachers, friends, and both direct and indirect contributors for their valuable insights, recommendations, and encouragement throughout this journey.

Abhishek Panthee	(THA080BCT002)
Alin Timelsana	(THA080BCT003)
Kiran Paudel	(THA080BCT019)

ABSTRACT

The Uptime Tracker is a web-based monitoring system designed to track website availability and performance metrics in real-time. In today's digital landscape, website downtime can significantly impact productivity, service delivery, and user trust. This project addresses the need for automated monitoring of college websites and web services by implementing a centralized tracking system similar to status.tcioe.edu.np and downdetector.com.

The system is built using Bun runtime with Next.js for the frontend and Elysia.js for the backend API, providing fast and efficient full-stack performance. PostgreSQL serves as the database for reliable data storage. The platform features user authentication, website registration and ownership management, automated ping checks at 5-minute intervals, and comprehensive analytics dashboards. It monitors HTTP/HTTPS availability, measures response times, and calculates hourly and daily average uptime statistics. The application supports both public and private monitoring modes, allowing website owners to manage visibility of their site's status.

The Uptime Tracker provides real-time status information, historical data visualization, and performance analytics that enable administrators to quickly identify and respond to downtime incidents. This transparency improves user experience and supports data-driven decisions for service reliability improvements. The project documentation is prepared using LaTeX for professional formatting.

Keywords—Website Monitoring, Uptime Tracking, Bun, Next.js, Elysia.js, PostgreSQL, Real-time Status, Performance Analytics, Web Application.

TABLE OF CONTENTS

DECLARATION . . . . . **i**

COPYRIGHT . . . . . **ii**

ACKNOWLEDGEMENT . . . . . **iii**

ABSTRACT . . . . . **iv**

LIST OF FIGURES . . . . . **viii**

LIST OF TABLES . . . . . **ix**

LIST OF ABBREVIATIONS . . . . . **ix**

LIST OF SYMBOLS . . . . . **x**

1. Introduction **1**

1.1. Background 1

1.2. Problem Statement 1

1.3. Objectives 2

1.4. Scope and Limitations 3

1.5. Report Organization 4

2. LITERATURE REVIEW **5**

2.1. Introduction to Website Monitoring 5

2.2. Importance of Uptime Monitoring 5

2.3. Existing Monitoring Solutions 6

2.4. Technical Approaches to Monitoring 7

2.5. Database Considerations 8

2.6. Modern Web Development Stack 9

2.7. Identified Research Gaps 9

3. REQUIREMENTS ANALYSIS **11**

3.1. Introduction 11

3.2. Functional Requirements 11

3.3. Non-Functional Requirements 13

3.4. System Requirements 15
3.5. Use Case Specifications 16
3.6. Requirements Traceability 17

4. SYSTEM ARCHITECTURE AND METHODOLOGY 18
4.1. Introduction 18
4.2. System Architecture Overview 18
4.3. Technology Stack 18
4.4. Database Design 20
4.5. API Design 23
4.6. Monitoring Service Design 23
4.7. User Interface Design 24
4.8. Development Methodology 25
4.9. Deployment Architecture 26

5. EXPECTED OUTCOMES 27
5.1. Introduction 27
5.2. System Deliverables 27
5.3. Expected Functional Outcomes 28
5.4. Documentation Deliverables 29
5.5. Learning Outcomes 29
5.6. Potential Impact 30
5.7. Success Criteria 30

6. CONCLUSION 31
6.1. Summary 31
6.2. Expected Contributions 31
6.3. Feasibility Assessment 31
6.4. Potential Challenges 32
6.5. Conclusion 32

APPENDICES 33

A. PROJECT BUDGET 33
A.1. Software and Development Tools 33
A.2. Hosting and Cloud Services 33
A.3. Documentation and Miscellaneous 34
A.4. Total Budget Summary 34

B. PROJECT TIMELINE 35
B.1. Project Schedule Overview 35

B.2. Gantt Chart 35
B.3. Milestone Summary 36
B.4. Risk Mitigation Timeline 36

REFERENCES 37

LIST OF FIGURES

4.1. High-Level System Architecture 19
4.2. Database Entity Relationship Diagram 20
4.3. Monitoring Service Workflow 24
4.4. Dashboard Interface Wireframe 25
B.1. Project Gantt Chart 36

LIST OF TABLES

2.1. Comparison of Website Monitoring Solutions 7
3.1. Hardware Requirements 15
3.2. Software Requirements 16
3.3. Requirements Traceability Matrix 17
4.1. Technology Stack Overview 19
4.2. Users Table Schema 21
4.3. Ownership Table Schema 21
4.4. Analytics Table Schema 21
4.5. Average Hour Table Schema 22
4.6. Average Day Table Schema 22
4.7. Refresh Tokens Table Schema 22
4.8. Password Reset Tokens Table Schema 23
4.9. Planned API Endpoints 23
5.1. Expected Monitoring Performance Metrics 28
5.2. Project Success Criteria 30
A.1. Development Tools (No Cost) 33
A.2. Hosting and Cloud Services Cost 33
A.3. Miscellaneous Expenses 34
A.4. Total Estimated Budget 34
B.1. Project Milestones 36

1. INTRODUCTION

1.1. Background

In the modern digital landscape, website availability has become a critical metric for organizations across all sectors. According to industry research, website downtime costs businesses an average of $5,600 per minute, with major enterprises potentially losing hundreds of thousands of dollars per hour of unavailability. Beyond financial implications, service disruptions erode user trust, damage brand reputation, and can lead to permanent customer loss in competitive markets.

Educational institutions have increasingly digitized their operations, relying on web-based systems for student information management, examination portals, library services, online learning platforms, and administrative functions. Thapathali Campus, like many academic institutions, operates multiple web services that students and faculty depend upon daily. When these services experience downtime, it affects academic activities, access to learning resources, and administrative processes. However, users often lack visibility into whether a service is experiencing issues, leading to repeated access attempts and frustration.

Website monitoring systems address this challenge by continuously checking service availability and providing real-time status information. Commercial solutions like Pingdom and UptimeRobot offer comprehensive monitoring features but require recurring subscription costs. Open-source alternatives like Uptime Kuma provide self-hosted options but may require technical expertise for deployment and maintenance. Platforms such as status.tcioe.edu.np demonstrate the value of institutional status pages, while crowd-sourced services like Downdetector aggregate user reports to identify widespread outages.

The need for an accessible, customizable, and institutionally-controlled monitoring solution motivates the development of the Uptime Tracker system. This project aims to create a web application that enables automated website monitoring, historical data analysis, and public status page generation using modern web technologies.

1.2. Problem Statement

Despite the critical importance of web service availability, many institutions including Thapathali Campus lack a centralized system to monitor and communicate the status of

their digital services. This absence creates several challenges:

- **Lack of proactive monitoring:** Without automated checks, downtime often goes undetected until users report issues, delaying response and extending service disruption periods.
- **No historical visibility:** System administrators lack data on uptime patterns, peak failure times, and response time trends, making it difficult to identify recurring issues or measure service improvements.
- **User uncertainty:** When accessing a slow or unresponsive website, users cannot determine whether the issue is with the service, their network connection, or a temporary glitch, leading to frustration and repeated access attempts.
- **No accountability metrics:** Without uptime records, institutions cannot measure service level compliance or demonstrate reliability improvements over time.
- **Decentralized information:** Status updates, when provided, are often scattered across social media, emails, or word-of-mouth, lacking a single authoritative source.

These challenges highlight the need for an integrated monitoring platform that provides automated availability checks, maintains historical records, and offers a public-facing status interface for transparency.

1.3. Objectives

The Uptime Tracker project aims to develop a comprehensive website monitoring system with the following objectives:

1.3.1. Primary Objectives

1. Design and implement a web application that enables users to register websites for automated uptime monitoring.
2. Develop a backend service that performs HTTP/HTTPS availability checks at regular 5-minute intervals and records response times.
3. Create a PostgreSQL database schema to store ping results, calculate hourly and daily averages, and maintain historical analytics data.
4. Build an interactive dashboard displaying real-time website status, response time graphs, and uptime statistics.
5. Implement user authentication and authorization to manage website ownership and visibility settings.

1.3.2. Secondary Objectives

1. Provide public status pages that allow anyone to view the availability of registered websites.
2. Enable website owners to control whether their monitoring data is publicly visible or private.
3. Generate aggregate statistics including average response times, uptime percentages, and trend analysis.
4. Ensure the system is deployable on standard infrastructure without requiring specialized hosting or expensive resources.

1.4. Scope and Limitations

1.4.1. Scope

The Uptime Tracker system encompasses the following features and components:

- **User Management:** Registration, authentication, and profile management for website owners.
- **Website Registration:** Interface for adding, editing, and removing websites from monitoring.
- **Automated Monitoring:** Background service performing HTTP/HTTPS checks every 5 minutes.
- **Data Storage:** PostgreSQL database storing raw ping results, hourly aggregates, and daily summaries.
- **Analytics Dashboard:** Interactive visualizations showing uptime history, response time trends, and current status.
- **Public Status Pages:** Publicly accessible pages displaying real-time status of opted-in websites.
- **Privacy Controls:** Options for website owners to make their monitoring data public or private.
- **Technology Stack:** Implementation using Bun runtime, Next.js frontend, Elysia.js backend API, and PostgreSQL database.

1.4.2. Limitations

The current version of the system operates within the following constraints:

- **Protocol Support:** Monitoring is limited to HTTP and HTTPS protocols. Other protocols such as FTP, SMTP, or custom TCP services are not supported.

- **Check Frequency:** The 5-minute monitoring interval represents a balance between detection speed and server resource consumption. Shorter intervals would increase database size and processing load.
- **Geographic Scope:** Availability checks originate from a single server location, so regional connectivity issues may not be detected if the monitoring server can reach the target.
- **Functionality Depth:** The system verifies that websites respond to HTTP requests but does not test specific application functionality, form submissions, or dynamic content rendering.
- **Alerting:** The initial version does not include email, SMS, or push notification alerts for downtime events.
- **Performance Metrics:** Detailed metrics such as page load time, DNS resolution time, and SSL handshake duration are not captured in the current implementation.

1.5. Report Organization

This proposal report is organized into the following chapters:

- **Chapter 2: Literature Review** examines existing website monitoring solutions, technical approaches, and identifies gaps that motivate this project.
- **Chapter 3: Requirements Analysis** defines functional and non-functional requirements along with system specifications.
- **Chapter 4: System Architecture and Methodology** describes the technical design, database schema, and development approach.
- **Chapter 5: Expected Outcomes** outlines the anticipated deliverables and success criteria for the project.
- **Chapter 6: Conclusion** summarizes the proposal and assesses project feasibility.

Additionally, the appendices include:

- **Appendix A: Project Budget** provides a detailed cost estimation for the project.
- **Appendix B: Project Timeline** presents the Gantt chart and milestone schedule.

2. LITERATURE REVIEW

2.1. Introduction to Website Monitoring

Website monitoring refers to the practice of systematically testing and verifying that websites and web services are functioning correctly and accessible to end users [1]. This process involves automated systems that periodically send requests to target URLs and analyze the responses to determine availability status. The importance of website monitoring has grown significantly as organizations increasingly depend on web-based services for critical operations.

Modern monitoring systems track multiple metrics including availability (whether the site responds), response time (how quickly it responds), and status codes (whether the response indicates success or error) [2]. These metrics provide administrators with the data needed to identify problems, measure service quality, and ensure compliance with service level agreements.

The evolution of monitoring has progressed from simple ping-based checks to sophisticated synthetic monitoring that simulates user interactions. However, for many use cases, basic HTTP availability monitoring provides sufficient visibility into service health without the complexity of full synthetic monitoring solutions.

2.2. Importance of Uptime Monitoring

Website downtime carries significant costs across multiple dimensions. Research by the Ponemon Institute indicates that IT downtime costs organizations an average of $5,600 per minute, though actual costs vary widely based on organization size and business model [3]. For e-commerce sites, downtime directly translates to lost sales. For educational institutions, downtime disrupts learning activities and administrative processes.

Beyond direct financial impact, downtime affects user perception and trust. Studies in user experience research demonstrate that users form lasting negative impressions from poor reliability experiences [4]. A website that is frequently unavailable or slow trains users to seek alternatives. This is particularly relevant for educational portals where students may have time-sensitive needs for accessing results, registration systems, or learning materials.

Proactive monitoring enables organizations to detect issues before users report them, often reducing mean time to resolution. Historical monitoring data also supports capacity planning, helps identify patterns that predict failures, and provides evidence for infrastructure investment decisions [5].

2.3. Existing Monitoring Solutions

2.3.1. Commercial Platforms

Several commercial platforms dominate the website monitoring market, each with distinct features and pricing models:

Pingdom [6] offers comprehensive monitoring from multiple geographic locations, synthetic transaction monitoring, and detailed performance analytics. The service provides real user monitoring capabilities and integration with popular notification services. However, pricing scales with the number of monitors and features required, making it potentially expensive for organizations monitoring many endpoints.

UptimeRobot [7] provides a popular freemium model with 5-minute check intervals on the free tier. The service monitors HTTP/HTTPS endpoints, provides status pages, and offers alerting through various channels. Its simplicity and free tier make it accessible for small projects, though advanced features require paid subscriptions.

StatusCake offers website monitoring with features including page speed monitoring, server monitoring, and domain expiration checking. The platform provides public status pages and supports integration with incident management tools.

Datadog provides enterprise-grade monitoring as part of a broader observability platform. While powerful, its complexity and cost position it for larger organizations with dedicated operations teams.

2.3.2. Open-Source Solutions

Open-source alternatives provide self-hosted options with different trade-offs:

Uptime Kuma has gained significant popularity as a self-hosted monitoring solution. Built with Node.js and featuring a clean web interface, it supports multiple monitor types including HTTP, TCP, DNS, and Docker containers. The project's active development and Docker deployment option make it accessible for users with basic technical skills. However, it requires server infrastructure to host and maintain.

Nagios represents the traditional enterprise monitoring approach with extensive plugin ecosystems and flexible configuration. Originally designed for infrastructure monitoring,

it can monitor websites through HTTP checks. However, its configuration complexity and dated interface present barriers for users without specialized experience.

Zabbix offers comprehensive monitoring capabilities including web scenarios that can test multi-step interactions. Like Nagios, it requires significant setup and expertise but provides powerful features for organizations willing to invest in configuration.

Prometheus with Blackbox Exporter enables HTTP probing within cloud-native environments. This approach integrates well with Kubernetes deployments and modern infrastructure but requires familiarity with the Prometheus ecosystem.

2.3.3. Comparison of Solutions

Table 2.1 summarizes key characteristics of existing monitoring solutions:

Table 2.1. *Comparison of Website Monitoring Solutions*

Solution	Type	Min Interval	Status Page	Self-Hosted
Pingdom	Commercial	1 minute	Yes	No
UptimeRobot	Freemium	5 minutes	Yes	No
StatusCake	Freemium	5 minutes	Yes	No
Uptime Kuma	Open-Source	20 seconds	Yes	Yes
Nagios	Open-Source	1 minute	Plugin	Yes
Zabbix	Open-Source	1 minute	Built-in	Yes

2.4. Technical Approaches to Monitoring

2.4.1. HTTP-Based Monitoring

HTTP monitoring sends requests to target URLs and analyzes the responses. A basic availability check verifies that the server responds with a successful status code (typically in the 200 range). More sophisticated checks may validate response content, check SSL certificate validity, or verify specific headers.

The HTTP protocol defines status codes that indicate request outcomes: 2xx codes indicate success, 3xx indicate redirects, 4xx indicate client errors, and 5xx indicate server errors. Monitoring systems interpret these codes to determine availability status. A 200 response generally indicates the site is up, while 5xx responses indicate server-side problems.

Response time measurement provides insight into performance beyond simple availability. This metric captures the total time from request initiation to response completion, including DNS resolution, TCP connection, TLS handshake, and data transfer. Slow response times

may indicate server overload, network issues, or application problems even when the site remains technically available.

2.4.2. Check Intervals and Data Volume

The frequency of monitoring checks involves trade-offs between detection speed and resource consumption. More frequent checks detect outages faster but generate more data and consume more resources on both the monitoring system and monitored targets.

A 5-minute interval represents a common balance point. With 5-minute checks, an outage will be detected within 5 minutes on average. This interval generates 288 data points per day per monitored site, a manageable volume for most database systems. Shorter intervals like 1 minute increase data volume fivefold while only marginally improving detection time for most practical purposes.

For systems monitoring many endpoints, check intervals also affect the load on the monitoring infrastructure itself. A system monitoring 100 websites at 1-minute intervals must handle 100 checks per minute, while 5-minute intervals reduce this to 20 checks per minute.

2.4.3. Data Aggregation Strategies

Raw monitoring data accumulates rapidly, making aggregation essential for long-term storage and efficient querying. Common aggregation strategies include:

Hourly aggregates summarize each hour's checks into average response time, minimum/maximum values, and availability percentage. This reduces 12 data points (at 5-minute intervals) to a single summary record while preserving meaningful performance information.

Daily aggregates further summarize 24 hourly records into daily statistics, enabling long-term trend analysis without storing indefinite raw data.

This hierarchical aggregation approach balances detail retention with storage efficiency. Recent data remains at full resolution for troubleshooting, while historical data is available in summarized form for trend analysis.

2.5. Database Considerations

Monitoring systems generate time-series data characterized by continuous timestamp-indexed measurements [8]. While specialized time-series databases like InfluxDB and TimescaleDB offer optimized storage and query capabilities for such data, traditional relational databases like PostgreSQL handle moderate monitoring workloads effectively [9].

PostgreSQL's temporal data types, window functions, and aggregation capabilities support common monitoring queries including average calculations, percentile analysis, and time-range filtering [10]. For systems monitoring hundreds rather than thousands of endpoints, PostgreSQL provides adequate performance with the benefits of relational integrity, mature tooling, and widespread familiarity.

The choice between specialized and general-purpose databases depends on scale requirements. A monitoring system tracking fewer than 1000 endpoints with 5-minute intervals generates approximately 288,000 records daily, well within PostgreSQL's capabilities. Larger deployments may benefit from purpose-built time-series solutions.

2.6. Modern Web Development Stack

The technology landscape for web application development has evolved significantly, with JavaScript/TypeScript ecosystems now supporting full-stack development [11]:

Next.js [12] provides a React-based framework for building web applications with server-side rendering, static generation, and API routes [13]. Its integrated development experience simplifies building interactive dashboards and data visualizations.

Bun [14] is a modern JavaScript runtime designed for speed, offering significantly faster startup times and package installation compared to Node.js. Its compatibility with npm packages and native TypeScript support make it attractive for new projects.

Elysia.js [15] is a TypeScript-first web framework built for Bun, emphasizing type safety and developer experience. Its end-to-end type inference enables catching errors at compile time rather than runtime.

These technologies represent a modern approach to web development that prioritizes developer productivity and application performance.

2.7. Identified Research Gaps

Analysis of existing solutions reveals gaps that the Uptime Tracker project addresses:

1. **Cost accessibility:** Commercial solutions require ongoing subscriptions that may exceed budgets for educational institutions or small organizations. Open-source alternatives are free but require infrastructure investment.
2. **Complexity barriers:** Enterprise tools like Nagios and Zabbix offer powerful capabilities but present steep learning curves. Simpler alternatives may lack features needed for meaningful monitoring.

3. **Deployment requirements:** Self-hosted solutions require Docker knowledge or server administration skills. A solution using familiar web development patterns could be more accessible.
4. **Customization needs:** Generic monitoring solutions may not address specific institutional requirements for status page branding, user management, or data retention policies.
5. **Modern technology adoption:** Many monitoring tools use older technology stacks. A solution built with current frameworks demonstrates modern development practices suitable for academic contexts.

The Uptime Tracker project aims to address these gaps by providing a straightforward, self-hosted monitoring solution built with modern web technologies, designed for deployment by users with standard web development experience.

3. REQUIREMENTS ANALYSIS

3.1. Introduction

Requirements analysis forms the foundation of software development by defining what the system must accomplish and the constraints within which it must operate. This chapter documents the functional requirements specifying system capabilities, non-functional requirements defining quality attributes, and technical requirements outlining the development and deployment environment. These requirements guide design decisions and provide criteria for evaluating the completed system.

3.2. Functional Requirements

Functional requirements specify the behaviors and features the Uptime Tracker system must provide to users. These requirements are organized by functional area.

3.2.1. User Management

1. **FR-U1: User Registration**
   The system shall allow new users to create accounts by providing a valid email address and password. Registration shall validate email format and enforce minimum password complexity requirements.
2. **FR-U2: User Authentication**
   The system shall authenticate users via email and password credentials. Upon successful authentication, the system shall issue a session token for subsequent requests.
3. **FR-U3: Session Management**
   The system shall maintain user sessions with configurable timeout periods. Users shall be able to explicitly log out, invalidating their session tokens.
4. **FR-U4: Password Security**
   The system shall store passwords using industry-standard hashing algorithms. Plain text passwords shall never be stored or logged.

3.2.2. Website Management

1. **FR-W1: Website Registration**
   Authenticated users shall be able to add websites for monitoring by providing a valid URL. The system shall validate URL format before accepting registration.
2. **FR-W2: Ownership Association**
   Each registered website shall be associated with the user who added it. Only the owner shall be able to modify or delete their website registrations.
3. **FR-W3: Visibility Control**
   Website owners shall be able to set their monitored websites as public or private. Public websites appear on status pages accessible to anyone, while private websites are visible only to their owners.
4. **FR-W4: Website Editing**
   Users shall be able to update the URL or visibility settings of their registered websites.
5. **FR-W5: Website Deletion**
   Users shall be able to remove websites from monitoring. Deletion shall remove the website record and associated monitoring data.

3.2.3. Monitoring Functionality

1. **FR-M1: Automated Checks**
   The system shall automatically perform availability checks on all registered websites at 5-minute intervals without manual intervention.
2. **FR-M2: HTTP/HTTPS Support**
   The system shall support monitoring of both HTTP and HTTPS URLs. For HTTPS URLs, the system shall verify that the site responds successfully.
3. **FR-M3: Response Time Measurement**
   Each availability check shall measure and record the response time in milliseconds from request initiation to response completion.
4. **FR-M4: Status Determination**
   The system shall determine website status based on HTTP response codes. Status codes 200-299 shall be considered successful (up), while other codes or connection failures shall be considered unsuccessful (down).
5. **FR-M5: Data Recording**
   Each check result shall be recorded with the website identifier, timestamp, response time, and success/failure status.

3.2.4. Data Aggregation

1. **FR-A1: Hourly Aggregation**
   The system shall calculate and store hourly statistics for each website including average response time and uptime percentage for each hour.
2. **FR-A2: Daily Aggregation**
   The system shall calculate and store daily statistics including average response time, minimum response time, maximum response time, and overall uptime percentage.
3. **FR-A3: Historical Retention**
   The system shall retain aggregated data for historical analysis. Raw ping data may be pruned after aggregation while aggregate data shall be retained long-term.

3.2.5. User Interface

1. **FR-I1: Dashboard View**
   Authenticated users shall have access to a dashboard displaying the current status of all their monitored websites at a glance.
2. **FR-I2: Status Indicators**
   The interface shall clearly indicate website status using visual cues (colors, icons) distinguishing between up, down, and unknown states.
3. **FR-I3: Response Time Display**
   The dashboard shall display current and historical response times with graphical visualizations showing trends over time.
4. **FR-I4: Uptime Statistics**
   The system shall display uptime percentages for configurable time periods (24 hours, 7 days, 30 days).
5. **FR-I5: Public Status Page**
   The system shall provide a publicly accessible status page showing the current status of all websites marked as public.

3.3. Non-Functional Requirements

Non-functional requirements define quality attributes and constraints that govern how the system performs its functions.

3.3.1. Performance Requirements

1. **NFR-P1: Response Time**
   Web pages and API endpoints shall respond within 500 milliseconds under normal load

conditions.

2. **NFR-P2: Concurrent Monitoring**
   The system shall support monitoring of at least 100 websites simultaneously without degraded performance.
3. **NFR-P3: Check Timing**
   Monitoring checks shall execute within the scheduled interval with no more than 30 seconds variance from the expected check time.

3.3.2. Reliability Requirements

1. **NFR-R1: System Availability**
   The monitoring service shall maintain at least 99% uptime, ensuring consistent check execution.
2. **NFR-R2: Data Integrity**
   All recorded monitoring data shall be accurately stored without loss or corruption.
3. **NFR-R3: Failure Recovery**
   The system shall recover automatically from transient failures and resume normal operation without manual intervention.

3.3.3. Security Requirements

1. **NFR-S1: Authentication Security**
   Passwords shall be hashed using bcrypt or equivalent algorithm with appropriate cost factor.
2. **NFR-S2: Session Security**
   Session tokens shall be generated using cryptographically secure random generation and transmitted only over HTTPS in production.
3. **NFR-S3: Data Access Control**
   Private monitoring data shall be accessible only to authenticated website owners. The system shall enforce authorization checks on all protected resources.
4. **NFR-S4: Input Validation**
   All user inputs shall be validated and sanitized to prevent injection attacks.

3.3.4. Usability Requirements

1. **NFR-U1: Responsive Design**
   The web interface shall be responsive and functional on desktop browsers, tablets, and mobile devices.

2. **NFR-U2: Intuitive Navigation**
   Users shall be able to accomplish primary tasks (view status, add website) with minimal navigation steps.
3. **NFR-U3: Clear Feedback**
   The interface shall provide clear feedback for user actions including success confirmations and error messages.

3.3.5. Maintainability Requirements

1. **NFR-M1: Code Organization**
   Source code shall be organized following established patterns with clear separation between frontend, backend, and database layers.
2. **NFR-M2: Documentation**
   Code shall include inline documentation for complex logic. API endpoints shall be documented with request/response specifications.

3.4. System Requirements

3.4.1. Hardware Requirements

Table 3.1 specifies the hardware requirements for running the Uptime Tracker system:

Table 3.1. *Hardware Requirements*

Component	Minimum	Recommended
Processor	64-bit dual-core CPU	64-bit quad-core CPU
Memory	4 GB RAM	8 GB RAM
Storage	10 GB available	50 GB SSD
Network	Stable internet connection	Low-latency broadband

3.4.2. Software Requirements

The following software components are required for development and deployment:

3.4.3. Development Tools

- **IDE:** Visual Studio Code with TypeScript extensions
- **Version Control:** Git for source code management
- **Package Manager:** Bun package manager (npm compatible)
- **Database Client:** pgAdmin or similar PostgreSQL management tool

Table 3.2. *Software Requirements*

Component	Specification
Operating System	Windows 10/11, macOS 12+, or Linux
Runtime	Bun 1.0 or later
Database	PostgreSQL 14 or later
Browser	Chrome, Firefox, Safari, or Edge (current versions)

3.5. Use Case Specifications

3.5.1. Use Case: Add Website for Monitoring

- **Actor:** Authenticated User
- **Precondition:** User is logged into the system
- **Main Flow:**
  1. User navigates to the Add Website page
  2. User enters the website URL
  3. User selects visibility setting (public/private)
  4. User submits the form
  5. System validates the URL format
  6. System creates ownership record
  7. System confirms successful addition
  8. Monitoring begins within the next check cycle
- **Postcondition:** Website is registered and will be monitored
- **Alternative Flow:** If URL is invalid, system displays error message and returns to step 2

3.5.2. Use Case: View Website Status

- **Actor:** Any User (public websites) or Authenticated User (private websites)
- **Precondition:** Website has been registered for monitoring
- **Main Flow:**
  1. User navigates to status page or dashboard
  2. System retrieves current status for requested websites
  3. System displays status indicators, response times, and uptime percentages
  4. User views historical data through graphs and statistics
- **Postcondition:** User has viewed website status information

3.6. Requirements Traceability

Table 3.3 maps high-level objectives to specific requirements:

Table 3.3. *Requirements Traceability Matrix*

Objective	Related Requirements
Automated monitoring	FR-M1, FR-M2, FR-M3, FR-M4, FR-M5
Historical data storage	FR-A1, FR-A2, FR-A3, NFR-R2
User dashboard	FR-I1, FR-I2, FR-I3, FR-I4, NFR-U1, NFR-U2
User authentication	FR-U1, FR-U2, FR-U3, NFR-S1, NFR-S2
Public status pages	FR-W3, FR-I5

4. SYSTEM ARCHITECTURE AND METHODOLOGY

4.1. Introduction

This chapter presents the architectural design of the Uptime Tracker system, detailing the component structure, data flow, database schema, and development methodology. The architecture follows a three-tier pattern separating presentation, business logic, and data storage concerns, enabling independent development and maintenance of each layer.

4.2. System Architecture Overview

The Uptime Tracker system employs a modern web application architecture consisting of four primary components:

1. **Frontend Application:** A Next.js-based web interface providing user interaction through dashboards, status pages, and management forms.
2. **Backend API:** An Elysia.js server handling business logic, authentication, and data operations through RESTful endpoints.
3. **Database:** PostgreSQL database storing user accounts, website registrations, and monitoring data.
4. **Monitoring Service:** A background process executing scheduled availability checks and recording results.

Figure 4.1 illustrates the high-level system architecture and component interactions.

4.3. Technology Stack

The technology stack for this project has been selected based on modern best practices, performance requirements, and development efficiency. Table 4.1 summarizes the key technologies:

4.3.1. Runtime Environment: Bun

Bun serves as the JavaScript/TypeScript runtime for both frontend and backend applications. Key characteristics include:

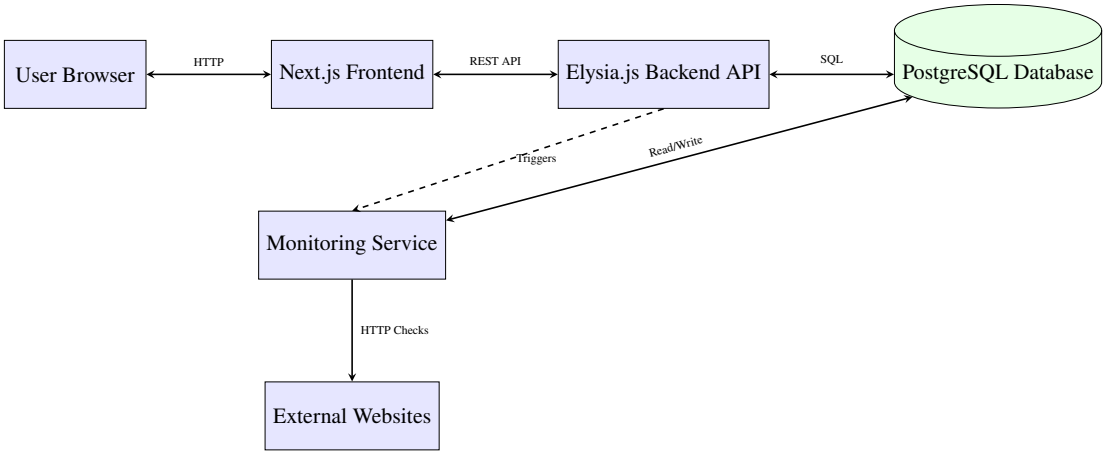

Figure 4.1. *High-Level System Architecture*

Table 4.1. *Technology Stack Overview*

Component	Technology	Purpose
Runtime	Bun	JavaScript/TypeScript execution
Frontend	Next.js	User interface and client-side logic
Backend	Elysia.js	REST API and business logic
Database	PostgreSQL	Data persistence and queries
Language	TypeScript	Type-safe development
Styling	Tailwind CSS	UI styling and responsiveness

- Native TypeScript support without separate compilation step
- Significantly faster package installation compared to npm
- Built-in bundler and test runner
- Drop-in compatibility with most Node.js packages
- Optimized performance for web server workloads

4.3.2. Frontend: Next.js

Next.js provides the foundation for the user interface with the following features utilized:

- **App Router:** File-based routing with layouts and nested routes
- **Server Components:** Reduced client-side JavaScript through server rendering
- **API Routes:** Backend-for-frontend patterns when needed
- **Built-in Optimization:** Automatic code splitting and image optimization

4.3.3. Backend: Elysia.js

Elysia.js handles API requests with these capabilities:

- **Type Safety:** End-to-end TypeScript inference from route definitions
- **Validation:** Built-in request validation using TypeBox schemas
- **Middleware:** Plugin system for authentication, logging, and CORS
- **Performance:** Optimized for Bun runtime with minimal overhead

4.3.4. Database: PostgreSQL

PostgreSQL provides reliable data persistence with:

- ACID compliance for data integrity
- Efficient temporal queries for time-series analysis
- Window functions for aggregation calculations
- JSON support for flexible data storage when needed
- Mature ecosystem with extensive tooling

4.4. Database Design

4.4.1. Entity Relationship Model

The database schema consists of seven primary tables managing users, authentication tokens, website ownership, and monitoring data. Figure 4.2 shows the entity relationships.

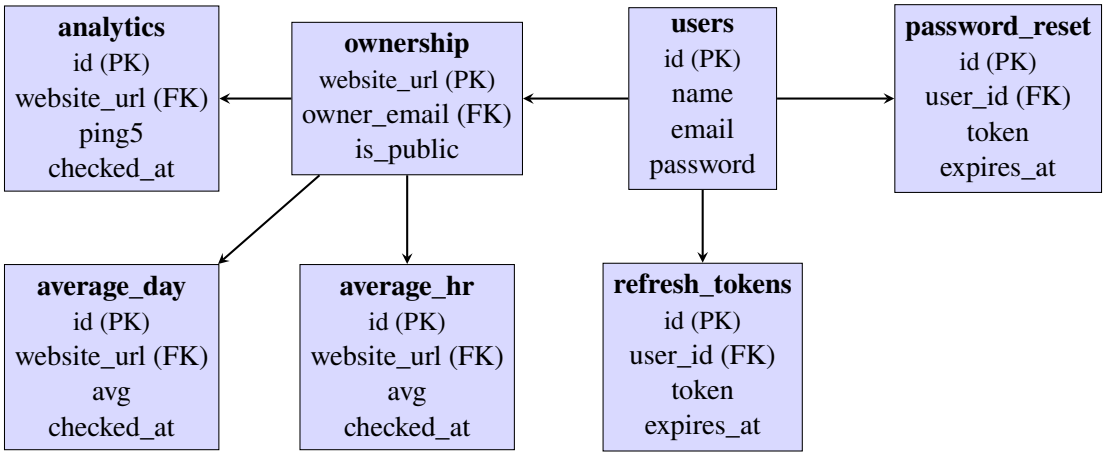

Figure 4.2. *Database Entity Relationship Diagram*

4.4.2. Table Specifications

Users Table

The users table stores account credentials and profile information:

Table 4.2. *Users Table Schema*

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing identifier
name	VARCHAR(50)	NOT NULL	User display name
email	VARCHAR(254)	UNIQUE, NOT NULL	User email address
password	VARCHAR(30)	NULL	User password
created_at	TIMESTAMP	DEFAULT NOW()	Account creation time

Ownership Table

The ownership table links users to their monitored websites:

Table 4.3. *Ownership Table Schema*

Column	Type	Constraints	Description
website_url	VARCHAR(254)	PRIMARY KEY	Website URL to monitor
owner_email	VARCHAR(254)	FOREIGN KEY	Reference to users email
is_public	BOOLEAN	DEFAULT TRUE	Public visibility flag

Analytics Table

The analytics table records individual ping results:

Table 4.4. *Analytics Table Schema*

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing identifier
website_url	VARCHAR(254)	FOREIGN KEY, NOT NULL	Reference to ownership
ping5	INTEGER	NULL	Response time in milliseconds
checked_at	TIMESTAMP	DEFAULT NOW()	Check timestamp

Average Hour Table

Stores hourly aggregated statistics:

Table 4.5. *Average Hour Table Schema*

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing identifier
website_url	VARCHAR(254)	FOREIGN KEY, NOT NULL	Reference to ownership
avg	INTEGER	NULL	Average response time
checked_at	TIMESTAMP	DEFAULT NOW()	Aggregation timestamp

Average Day Table

Stores daily aggregated statistics:

Table 4.6. *Average Day Table Schema*

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing identifier
website_url	VARCHAR(254)	FOREIGN KEY, NOT NULL	Reference to ownership
avg	INTEGER	NULL	Daily average response time
checked_at	TIMESTAMP	DEFAULT NOW()	Aggregation timestamp

Refresh Tokens Table

Stores JWT refresh tokens for authentication:

Table 4.7. *Refresh Tokens Table Schema*

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing identifier
user_id	INTEGER	FOREIGN KEY, NOT NULL	Reference to users
token	VARCHAR(500)	NOT NULL	Refresh token string
expires_at	TIMESTAMP	NOT NULL	Token expiration time
created_at	TIMESTAMP	DEFAULT NOW()	Token creation time
revoked_at	TIMESTAMP	NULL	Token revocation time

Password Reset Tokens Table

Stores password reset tokens for account recovery:

Table 4.8. *Password Reset Tokens Table Schema*

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing identifier
user_id	INTEGER	FOREIGN KEY, NOT NULL	Reference to users
token	VARCHAR(255)	NOT NULL	Reset token string
expires_at	TIMESTAMP	NOT NULL	Token expiration time
created_at	TIMESTAMP	DEFAULT NOW()	Token creation time
used_at	TIMESTAMP	NULL	Token usage time

4.5. API Design

The backend will expose RESTful API endpoints organized by resource type. Table 4.9 summarizes the planned endpoints:

Table 4.9. *Planned API Endpoints*

Method	Endpoint	Description
POST	/api/auth/register	Create new user account
POST	/api/auth/login	Authenticate and receive token
POST	/api/auth/logout	Invalidate session
GET	/api/websites	List user's monitored websites
POST	/api/websites	Add new website to monitor
PUT	/api/websites/:id	Update website settings
DELETE	/api/websites/:id	Remove website from monitoring
GET	/api/status/:url	Get current status of website
GET	/api/analytics/:url	Get historical data for website
GET	/api/public/status	Get all public website statuses

4.6. Monitoring Service Design

4.6.1. Check Execution Flow

The monitoring service will operate as a background process executing the following workflow:

1. **Schedule Trigger:** A scheduler triggers checks every 5 minutes (configurable via cron or interval timer).
2. **URL Retrieval:** The service queries all registered URLs from the ownership table.
3. **Parallel Execution:** HTTP requests are sent to all URLs concurrently with connection and response timeouts.
4. **Result Recording:** For each URL, the service records response time (or null for failures) and status in the analytics table.
5. **Aggregation Update:** After recording raw results, the service updates hourly aggregates as needed.

Figure 4.3 illustrates the monitoring service workflow.

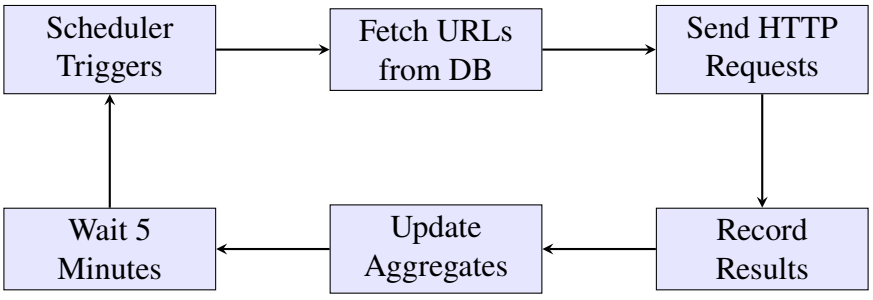

Figure 4.3. *Monitoring Service Workflow*

4.7. User Interface Design

4.7.1. Component Structure

The frontend follows a component-based architecture with these primary views:

- **Landing Page:** Public introduction and login/register options
- **Dashboard:** Overview of all user's monitored websites with status indicators
- **Website Detail:** Historical charts and statistics for individual websites
- **Add Website:** Form for registering new websites to monitor
- **Public Status Page:** Grid showing all public website statuses
- **Settings:** User profile and account management

4.7.2. Dashboard Layout

The dashboard will display website cards in a responsive grid, each card showing:

- Website URL and friendly name
- Current status indicator (green/red/gray)
- Latest response time
- 24-hour uptime percentage
- Sparkline chart of recent response times

Figure 4.4 shows the proposed dashboard interface wireframe.

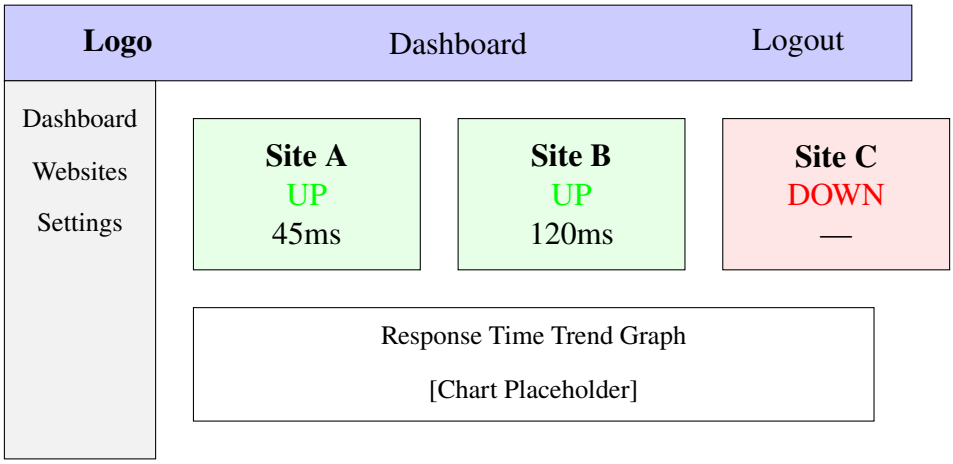

Figure 4.4. *Dashboard Interface Wireframe*

4.8. Development Methodology

4.8.1. Agile Approach

The project follows an iterative development methodology with these characteristics:

- **Incremental Development:** Features are implemented in small, functional increments
- **Regular Integration:** Code is frequently committed and integrated to identify issues early
- **Prioritized Backlog:** Features are prioritized by core functionality first, enhancements later
- **Testing Integration:** Each feature includes corresponding tests before moving to next

4.8.2. Development Phases

1. **Phase 1 - Foundation:** Project setup, database schema, basic authentication
2. **Phase 2 - Core Features:** Website registration, monitoring service, data recording
3. **Phase 3 - Interface:** Dashboard implementation, status displays, visualizations
4. **Phase 4 - Polish:** Public status pages, aggregation, error handling, testing
5. **Phase 5 - Documentation:** Code documentation, user guide, deployment instructions

4.8.3. Version Control

Git will be used for version control with the following practices:

- Feature branches for isolated development
- Descriptive commit messages documenting changes
- Regular commits maintaining manageable change sets
- Main branch always in deployable state

4.9. Deployment Architecture

The system is designed for deployment on standard infrastructure:

- **Single Server:** All components can run on one server for simple deployments
- **Separate Services:** Frontend and backend can be deployed separately for scaling
- **Database Hosting:** PostgreSQL can be self-hosted or use managed services
- **Process Management:** Background monitoring service managed via process managers

Minimum production requirements include a Linux server with Bun installed, PostgreSQL database access, and domain configuration for HTTPS termination.

5. EXPECTED OUTCOMES

5.1. Introduction

This chapter outlines the anticipated results and deliverables upon successful completion of the Uptime Tracker project. The expected outcomes are derived from the objectives defined in Chapter 1 and the system design presented in Chapter 4.

5.2. System Deliverables

Upon completion, the project will deliver the following tangible outputs:

5.2.1. Web Application

A fully functional web application accessible via modern web browsers with the following capabilities:

1. **User Management Module**
   - Secure user registration with email verification
   - Login/logout functionality with session management
   - Password reset capability
   - User profile management
2. **Website Monitoring Module**
   - Interface to add, edit, and remove websites from monitoring
   - Real-time status display showing current availability
   - Response time visualization through interactive charts
   - Historical uptime statistics and trends
3. **Public Status Page**
   - Publicly accessible page showing status of opted-in websites
   - Clean, minimal interface requiring no authentication
   - Auto-refresh functionality for live updates

5.2.2. Backend Services

The project will deliver a robust backend system comprising:

1. **RESTful API** providing endpoints for all application functionality
2. **Monitoring Service** executing automated availability checks every 5 minutes
3. **Data Aggregation Service** computing hourly and daily statistics
4. **Authentication Service** managing JWT-based user sessions

5.2.3. Database System

A PostgreSQL database storing:

- User account information
- Website registration and ownership records
- Raw ping result data (response time, status, timestamp)
- Computed hourly and daily aggregates
- System configuration parameters

5.3. Expected Functional Outcomes

5.3.1. Monitoring Accuracy

The system is expected to achieve the following monitoring performance:

Table 5.1. *Expected Monitoring Performance Metrics*

Metric	Expected Value
Check Interval Accuracy	± 30 seconds of scheduled time
Response Time Measurement	± 10ms variance
Downtime Detection	Within 5-10 minutes
Data Storage Reliability	99.9% data integrity
API Response Time	< 200ms for standard queries

5.3.2. User Experience

The application interface is expected to provide:

- Intuitive navigation requiring minimal learning curve
- Responsive design functioning on desktop, tablet, and mobile devices

- Clear visual indicators for website status (up/down)
- Informative error messages guiding user actions
- Fast page load times (< 2 seconds on standard connections)

5.3.3. System Scalability

The designed architecture should support:

- Up to 100 concurrent users without performance degradation
- Monitoring of up to 500 websites from a single server instance
- Storage of 6 months of historical data per website
- Horizontal scaling capability for future growth

5.4. Documentation Deliverables

The project will produce comprehensive documentation including:

1. **Technical Documentation**
   - Database schema documentation with entity relationships
   - API endpoint reference with request/response formats
   - System architecture diagrams
   - Deployment and configuration guide
2. **User Documentation**
   - User manual for website owners
   - Administrator guide for system management
   - Troubleshooting guide for common issues
3. **Academic Documentation**
   - Project proposal (current document)
   - Final project report with implementation details
   - Presentation materials for defense

5.5. Learning Outcomes

Through the development of this project, the team expects to gain practical experience in:

- Full-stack web application development using modern technologies

- Database design and optimization for time-series data
- RESTful API design and implementation
- Background service development and scheduling
- User authentication and security best practices
- Version control and collaborative development workflows
- Technical documentation and report writing

5.6. Potential Impact

Upon successful implementation, the Uptime Tracker system is expected to provide:

1. **For Thapathali Campus:** A functional tool to monitor institutional web services, improving awareness of service availability and enabling proactive maintenance.
2. **For Students and Faculty:** A transparent status page reducing uncertainty about service availability and providing a reference for checking institutional website status.
3. **For System Administrators:** Historical data and analytics to identify patterns, optimize maintenance schedules, and demonstrate service reliability metrics.
4. **As a Reference Implementation:** A documented open-source project that can serve as a learning resource for future DBMS and web development projects.

5.7. Success Criteria

The project will be considered successful if it meets the following criteria:

Table 5.2. *Project Success Criteria*

Criterion	Acceptance Condition
Core Functionality	All primary objectives implemented and operational
Monitoring Reliability	Automated checks execute without manual intervention
Data Integrity	No data loss during normal operation
User Interface	Dashboard displays accurate real-time status
Public Access	Status page accessible without authentication
Documentation	Complete technical and user documentation

6. CONCLUSION

6.1. Summary

This proposal presents the Uptime Tracker project, a web-based monitoring system designed to track website availability, measure response times, and provide analytics through an intuitive dashboard. The project addresses a practical need within educational institutions and organizations for transparent service status monitoring without the recurring costs associated with commercial solutions.

The proposed system employs a modern three-tier architecture with a Next.js frontend, Elysia.js backend API, and PostgreSQL database, all running on the Bun JavaScript runtime. This technology stack demonstrates contemporary full-stack development practices while maintaining simplicity suitable for deployment on standard infrastructure.

6.2. Expected Contributions

Upon successful completion, the project is expected to deliver:

1. A functional web application enabling automated website monitoring with 5-minute check intervals
2. A comprehensive database system storing raw ping results, hourly aggregates, and daily statistics
3. An interactive dashboard displaying real-time website status and historical trends
4. Public status pages allowing transparent service health communication
5. Complete technical documentation including database schema, API reference, and deployment guide

6.3. Feasibility Assessment

The project is considered feasible based on the following factors:

- **Technical Feasibility:** All proposed technologies are well-documented, actively maintained, and have been successfully used in similar applications. The development team has prior experience with JavaScript/TypeScript web development.

- **Time Feasibility:** The 3-month timeline allows adequate time for design, development, testing, and documentation phases with buffer time for unexpected challenges.
- **Resource Feasibility:** The project requires minimal financial investment as all development tools are open-source. Free-tier cloud services can support initial deployment.
- **Scope Appropriateness:** The defined scope is realistic for a DBMS academic project, focusing on core monitoring functionality without overambitious feature expansion.

6.4. Potential Challenges

The following challenges are anticipated and will be addressed during development:

1. **Reliable Scheduling:** Ensuring the monitoring service executes checks consistently requires careful implementation of the scheduling mechanism and error handling.
2. **Database Optimization:** As monitoring data accumulates, query performance may degrade. Proper indexing and data archival strategies will be implemented.
3. **Authentication Security:** Implementing secure user authentication requires careful attention to password hashing, token management, and session handling.
4. **Real-time Updates:** Displaying live status updates on the dashboard without excessive server load requires efficient data fetching strategies.

6.5. Conclusion

The Uptime Tracker project represents a practical application of database management concepts combined with modern web development practices. By developing this system, the team will gain hands-on experience with full-stack development, database design, API implementation, and background service development.

The proposed system fills a genuine need for accessible website monitoring within educational and organizational contexts. Upon completion, the Uptime Tracker will provide Thapathali Campus with a tool for monitoring institutional web services while serving as a reference implementation for future academic projects.

The project team is confident in the successful completion of this project within the proposed timeline and budget, delivering a functional, documented, and deployable website monitoring system.

A. PROJECT BUDGET

The total budget required for the successful completion of the Uptime Tracker project is estimated to be between NPR 5,000 and NPR 10,000. Since this is a software-based project, the costs are primarily associated with cloud services, domain registration, and miscellaneous documentation expenses.

A.1. Software and Development Tools

All development tools used in this project are open-source and free:

Table A.1. *Development Tools (No Cost)*

Tool	Description	Cost
Bun Runtime	JavaScript/TypeScript runtime	Free
Next.js	React framework for frontend	Free
Elysia.js	Backend web framework	Free
PostgreSQL	Open-source database	Free
Visual Studio Code	Code editor	Free
Git	Version control system	Free
Tailwind CSS	CSS framework	Free
	Subtotal	**Free**

A.2. Hosting and Cloud Services

For deployment and hosting, the following services may be required:

Table A.2. *Hosting and Cloud Services Cost*

Service	Description	Estimated Cost
Domain Registration	.com or .np domain (1 year)	NPR 1,500 - 2,500
VPS Hosting	Basic cloud server (3 months)	NPR 2,000 - 4,000
PostgreSQL Hosting	Managed database (optional)	Free tier available
SSL Certificate	Let's Encrypt	Free
	Subtotal	**NPR 3,500 - 6,500**

Note: Free-tier options from Vercel (frontend), Railway (backend), and Supabase (database) can reduce hosting costs to zero during development and initial deployment.

A.3. Documentation and Miscellaneous

Table A.3. *Miscellaneous Expenses*

Item	Description	Estimated Cost
Report Printing	Proposal and final report printing	NPR 500 - 1,000
Report Binding	Spiral/hard binding	NPR 300 - 500
Stationery	Paper, pens, notebooks	NPR 200 - 300
Internet	Development period data usage	NPR 500 - 700
	Subtotal	**NPR 1,500 - 2,500**

A.4. Total Budget Summary

Table A.4. *Total Estimated Budget*

Category	Estimated Cost
Development Tools	Free
Hosting and Cloud Services	NPR 3,500 - 6,500
Documentation and Miscellaneous	NPR 1,500 - 2,500
Total Estimated Cost	**NPR 5,000 - 9,000**

Note: The actual costs may vary depending on the hosting provider chosen and the duration of service usage. Using free-tier cloud services can significantly reduce the overall budget.

B. PROJECT TIMELINE

The project will be executed over a period of approximately 3 months, divided into several key phases. Each phase includes specific tasks and milestones to ensure steady progress toward project completion.

B.1. Project Schedule Overview

The project timeline spans from December 12, 2025 to March 31, 2026, covering the following major phases:

1. **Project Initiation (Dec 12 - Dec 20):** Topic selection, problem definition, and literature review
2. **System Design (Dec 18 - Dec 30):** Requirements analysis, database design, and architecture design
3. **Proposal Preparation (Dec 28 - Jan 2):** Proposal writing and submission
4. **Proposal Submission:** January 2, 2026
5. **Development (Jan 3 - Mar 10):** Backend development, frontend development, monitoring service implementation, and system integration
6. **Testing & Documentation (Mar 5 - Mar 28):** Testing, bug fixes, deployment, and final report preparation
7. **Final Defense:** March 31, 2026

B.2. Gantt Chart

The Gantt chart in Figure B.1 provides a visual representation of the project timeline, showing task durations, dependencies, and key milestones.

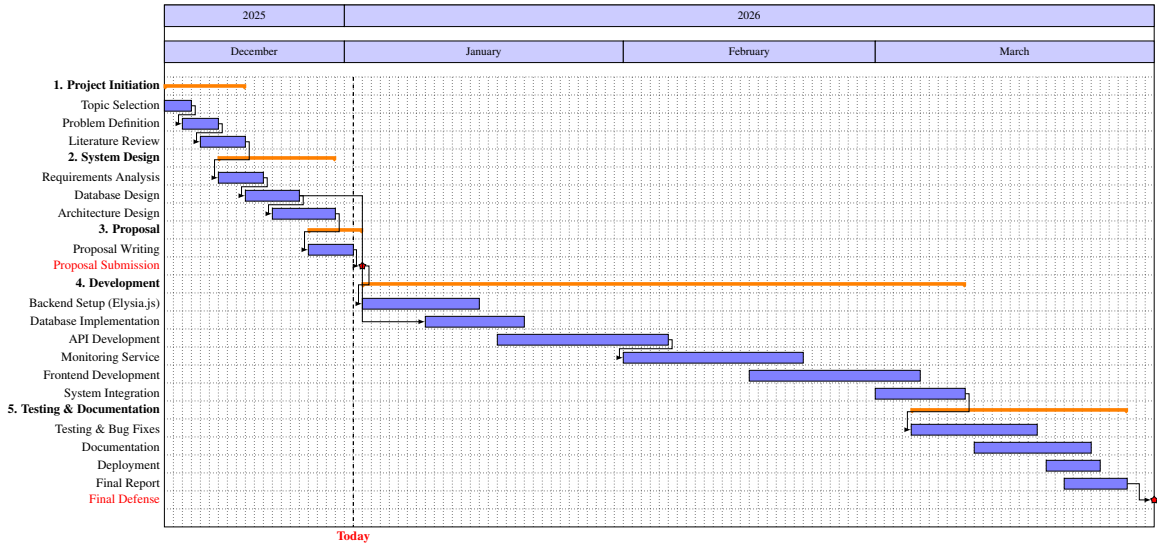

Figure B.1. *Project Gantt Chart*

B.3. Milestone Summary

Key project milestones are summarized in Table B.1:

Table B.1. *Project Milestones*

Milestone	Target Date	Deliverable
Project Start	December 12, 2025	Topic selection and planning
Design Complete	December 30, 2025	Database schema and architecture
Proposal Submission	January 2, 2026	Complete project proposal
Backend Complete	February 15, 2026	Working API and monitoring service
Development Complete	March 10, 2026	Fully integrated application
Final Report	March 28, 2026	Complete project documentation
Final Defense	March 31, 2026	Project presentation and demo

B.4. Risk Mitigation Timeline

Buffer time has been allocated for potential delays:

- 3-day buffer between major development phases
- Parallel documentation to avoid end-of-project crunch
- Early integration testing to identify issues before final phase

REFERENCES

[1] I. Sommerville, *Software Engineering*, 10th ed. Boston: Pearson, 2016.

[2] T. Berners-Lee, R. Fielding, and H. Frystyk, "Hypertext transfer protocol – http/1.0," *RFC 1945, IETF*, 1996.

[3] Ponemon Institute, "Cost of data center outages," *Ponemon Institute Research Report*, 2016, sponsored by Emerson Network Power.

[4] J. Nielsen, "Response times: The 3 important limits," in *Usability Engineering*. Morgan Kaufmann, 1994, available at Nielsen Norman Group.

[5] Gartner Research, "The cost of downtime," *Gartner Research Note*, 2014.

[6] Pingdom, "Website monitoring - uptime monitoring," https://www.pingdom.com/, 2023, accessed: 2025-12-15.

[7] UptimeRobot, "Uptimerobot - free website monitoring service," https://uptimerobot.com/, 2023, accessed: 2025-12-15.

[8] C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston: Addison-Wesley, 2003.

[9] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*. The PostgreSQL Global Development Group, 2024, available at https://www.postgresql.org/docs/16/.

[10] E. F. Codd, "A relational model of data for large shared data banks," in *Communications of the ACM*, vol. 13, no. 6, 1970, pp. 377–387.

[11] Microsoft, "Typescript - javascript with syntax for types," https://www.typescriptlang.org/, 2024, accessed: 2025-12-15.

[12] Vercel, "Next.js - the react framework for production," https://nextjs.org/, 2024, accessed: 2025-12-15.

[13] Meta Platforms, "React: A javascript library for building user interfaces," *React Documentation*, 2020, available at https://react.dev/.

[14] Oven, "Bun - javascript runtime and toolkit," https://bun.sh/, 2024, accessed: 2025-12-15.

[15] SaltyAom, “Elysia - ergonomic framework for humans,” https://elysiajs.com/, 2024, accessed: 2025-12-15.